gems/activesupport-4.1.0/lib/active_support/concern.rb
```
module ActiveSupport
  module Concern
    class MultipleIncludedBlocks < StandardError #:nodoc:
      def initialize
        super "Cannot define multiple 'included' blocks for a Concern"
      end
    end

    def self.extended(base)
      base.instance_variable_set(:@_dependencies, [])
    end

    # ...
```

When a module extends Concern, Ruby calls the extended *Hook Method (157)*, and extended defines an @_dependencies *Class Instance Variable (109)* on the includer. I'll show you what happens to this variable in a few pages. For now, just remember that all concerns have it, and it's initially an empty array.

To introduce Concern#append_features, the other important method in Concern, let me take you on a very short side-trip into Ruby's standard libraries.

Module#append_features

Module#append_features is a core Ruby method. It's similar to Module#included, in that Ruby will call it whenever you include a module. However, there is an important difference between append_features and included: included is a Hook Method that is normally empty, and it exists only in case you want to override it. By contrast, append_features is where the real inclusion happens. append_features checks whether the included module is already in the includer's chain of ancestors, and if it's not, it adds the module to the chain.

There is a reason why you didn't read about append_features in the first part of this book: in your normal coding, you're supposed to override included, not append_features. If you override append_features, you can get some surprising results, as in the following example:

part2/append_features.rb
```
module M
  def self.append_features(base); end
end

class C
  include M
end

C.ancestors   # => [C, Object, Kernel, BasicObject]
```